

BPL_YEAST_AIR_Fedbatch - demo

Author: Jan Peter Axelsson

This notebook demonstrate yeast fedbatch cultivation. We look at impact of changes in the glucose feeding. We also take a look at tuning of the DO-control system. Both liquid- and gasphase are included in the model. The culture growth and metabolism are formulated in relation to the cells respiratory capacity [1] as well as oxygen limitation in the reactor. The model is expanded to describe also the gas phase as well as the culture heat production [2]. The model was derived mainly from continuous culture data but proved to capture dynamic aspects well of ethanol production and consumption [3]. Several cultivations were done with ethanol control that facilitated high cell density cultivations [4].

Interaction with the compiled model as FMU is mainly through the simplified commands: `par()`, `init()`, `newplot()`, `simu()` etc. The last simulation is always available in the workspace and called 'sim_res'. The command `describe()` brings mainly up description information from the actual Modelica code from the FMU but is complemented with information given in the dedicated Python setup-file.

The idea is to demonstrate how simulations and varying conditions can provide some process insight that can support the experimental work. I hope that at the end of this session you are ready to formulate your own questions you want to address with simulations - and you can just go on in this notebook! Just press the field "+Code" in the upper left part of notebook interface and you get a new "cell" where you write your own code. You can copy and paste from cells above using `ctrl-c` and `ctrl-p` as usual and edit the cell. When you are ready to execute the cell just press the "play button" to the left in the cell or press `shift-enter` as in "ordinary" Jupyter notebooks.

After a session you may want to save your own notebook. That you can do on your Google Drive account and I refer to Colab instructions for how to do this. It is easy.

Enjoy!

```
In [1]: # Import the application model and context variables
        from BPL_YEAST_AIR_Fedbatch_DOcontrol import*
```

Windows - run FMU pre-compiled JModelica 2.14

```
In [2]: # Import the general FMU_explore functions and adapt them to the application
        from fmu_explore_pyfmi import fmu_explore

        Dummy = fmu_explore(model, parValue, parLocation, parCheck, fmu_model, fmu_proc,
                             MSL_usage, MSL_version, BPL_version, \
                             opts_std, simulationTime, timeDiscreteStates, stateValue, \
                             diagrams, ax, lines)

        par = Dummy.par
        init = Dummy.init
        disp = Dummy.disp
```

```

simu = Dummy.simu
show = Dummy.show
resetPen = Dummy.resetPen
process_diagram = Dummy.process_diagram
describe_general = Dummy.describe_general
describe_parts = Dummy.describe_parts
describe_MSL = Dummy.describe_MSL
FMU_explore_info = Dummy.FMU_explore_info
system_info = Dummy.system_info
SDG = Dummy.SDG

```

Bring in the user defined FMU_explore functions describe() and newplot()

```
In [3]: run -i BPL_YEAST_AIR_Fedbatch_D0control_explore.py
```

Model for the process has been setup. Key commands:

- par() - change of parameters and initial values
- init() - change initial values only
- simu() - simulate and plot
- newplot() - make a new plot
- show() - show plot from previous simulation
- disp() - display parameters and initial values from the last simulation
- describe() - describe culture, broth, parameters, variables with values/units

Note that both disp() and describe() takes values from the last simulation and the command process_diagram() brings up the main configuration

Brief information about a command by help(), eg help(simu)

Key system information is listed with the command system_info()

```
In [4]: plt.rcParams['figure.figsize'] = [36/2.54, 30/2.54]
```

About the process model

We can get information about the process, liquid- and gas-phase by the command describe(). This command can also be used to bring up information about a specific variable or parameter. However, you should use describe() after a simulation to get the valued used during the simulation.

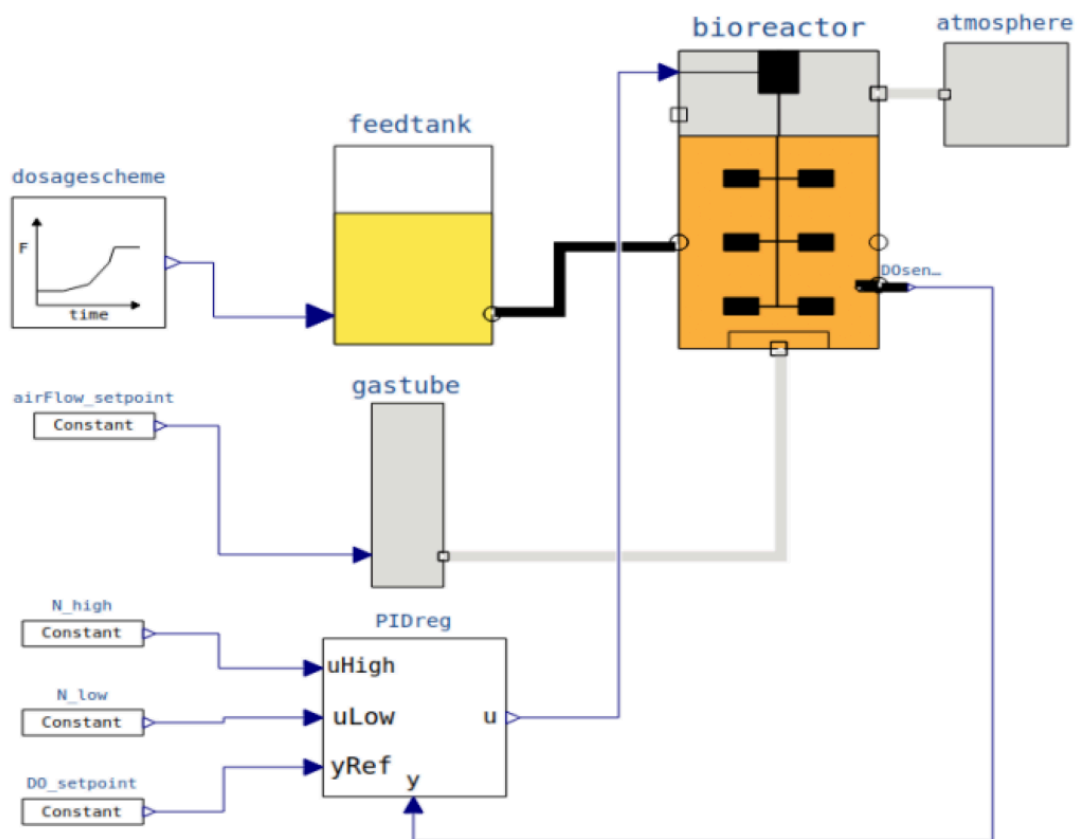
```
In [5]: describe('culture'); #print(); describe('Liquidphase'); print(); describe('gasph
```

Saccharomyces cerevisiae - default parameters for strain H1022

The model of the process has parameters both for culture, gas_liquid_transfer, as well as feeding procedure. The paramters that are available for changes you find by the command disp() and you get a long list and you change by them by command par(). The model has even more parameters in the background but not made available for interaction.

```
In [6]: process_diagram()
```

No processDiagram.png file in the FMU, but try the file on disk.



In [7]: `describe('process')`

In [8]: `describe('culture')`

Saccharomyces cerevisiae - default parameters for strain H1022

First simulations - adjusting start of substrate feeding

```
In [9]: # Culture parameters and others at default values
par(qO2max=0.0069)

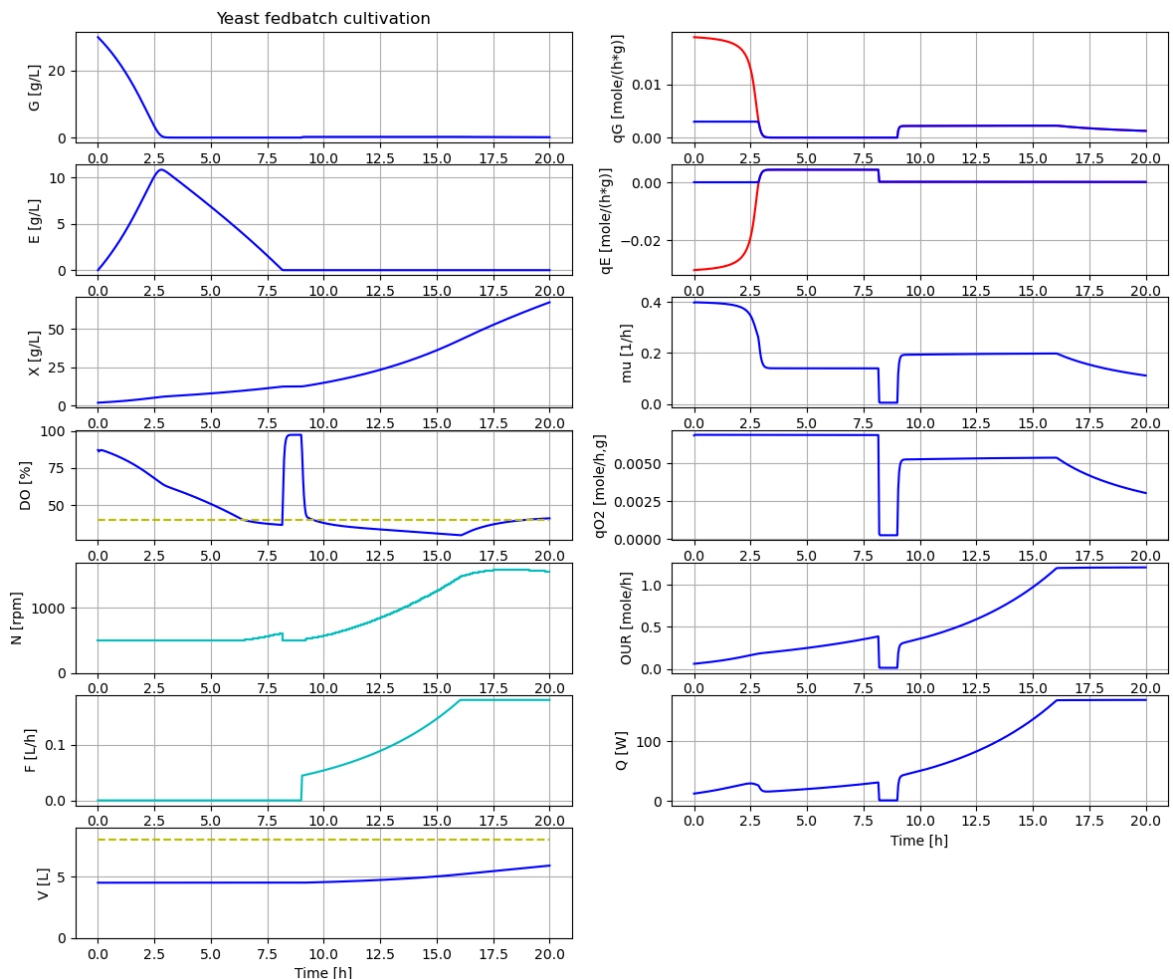
# Process initial conditions
init(V_start=4.5, VG_start=4.5*30, VX_start=4.5*2, VE_start=4.5*0)

# Feed profile
par(t_startExp=9, F_startExp=0.044, mu_feed=0.20, F_max=0.18)

# DO-control parameters
#par(samplePeriod=1/60, K=10, Ti=0.5, I_0=500)
par(K=10, Ti=0.5, I_start=0)

# Simulate and plot
newplot(title='Yeast fedbatch cultivation', plotType='Overview')
simu(20)
```

Out[9]: <pyfmi.fmi_algorithm_drivers.FMIResult at 0x2999696e570>



Now we can get value of broth volume as well as the headspace and values are the last ones in the simulation

```
In [10]: describe('bioreactor.V')
```

Reactor broth volume : 5.892 [L]

```
In [11]: describe('bioreactor.V_gasphase')
```

Volume of the gas phase : 2.108 [L]

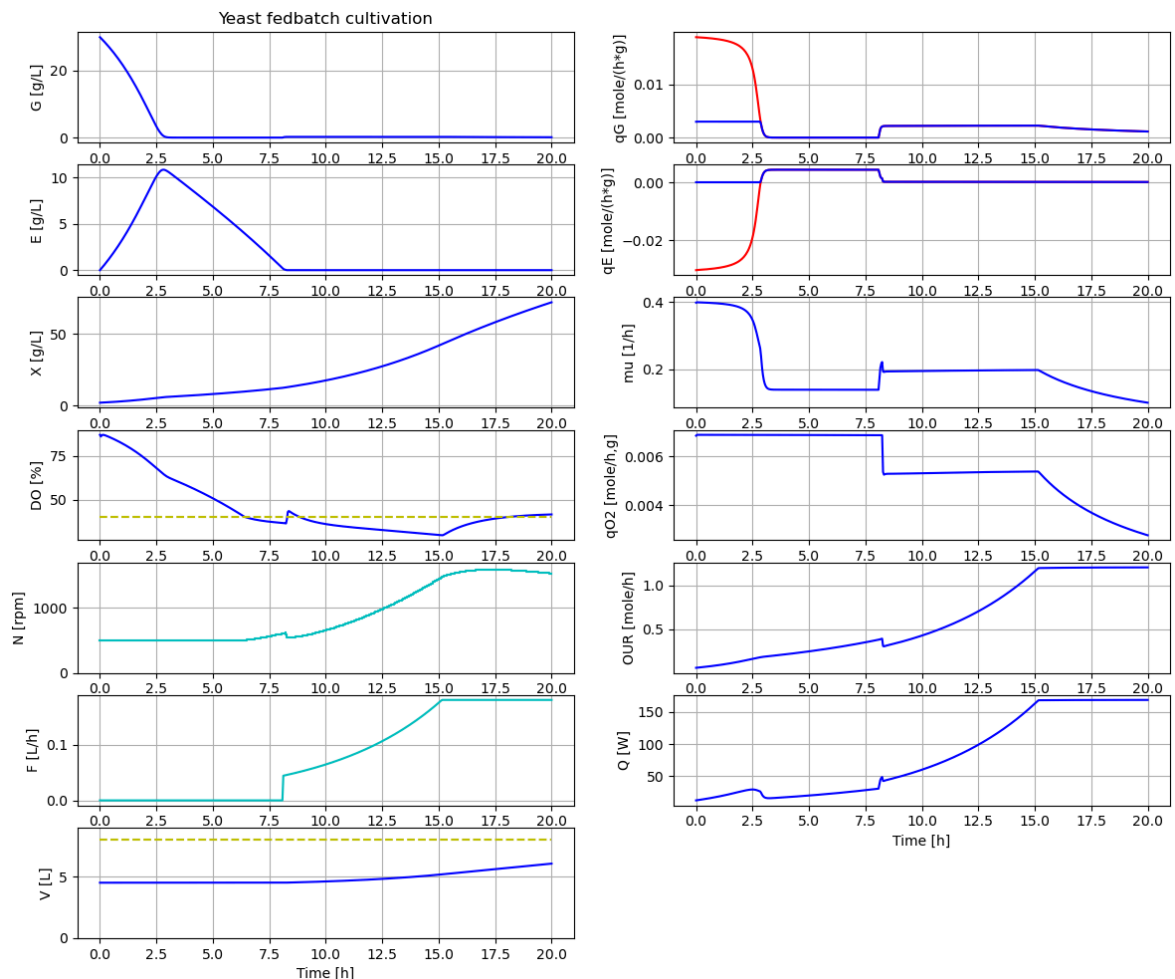
```
In [12]: # Take a look at the parameters available to adjust the dosage scheme
disp('dosage', decimals=4)
```

```
F_start : 0.0
mu_feed : 0.2
t_startExp : 9.0
F_startExp : 0.044
F_max : 0.18
```

```
In [13]: # Let us start the feeding just after the batch phase has ended and keep other p
par(t_startExp=8.1)

# Simulate and plot
newplot(title='Yeast fedbatch cultivation', plotType='Overview')
simu(20)
```

```
Out[13]: <pyfmi.fmi_algorithm_drivers.FMIResult at 0x29996c56c00>
```



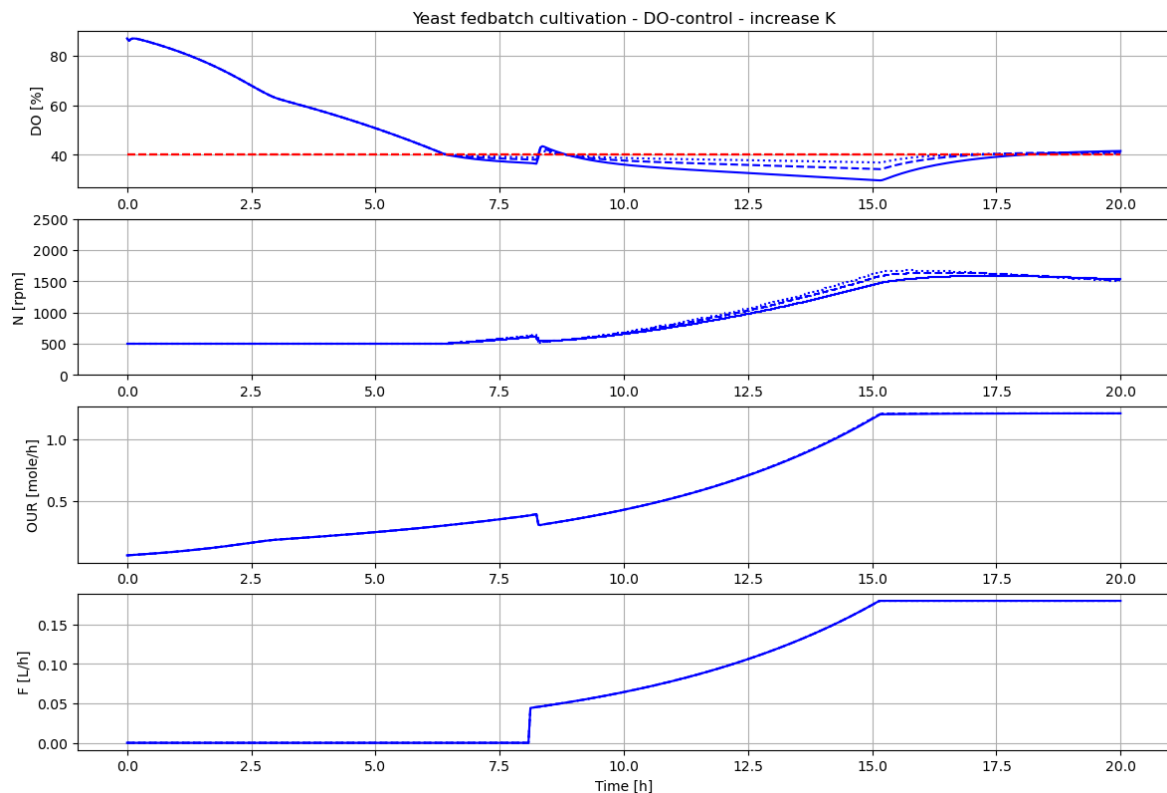
The increase of DO to about 50 % at end of batch phase should be possible to detect easily. This simulation is more realistic and we use these settings from now on.

DO-control - tuning of PI-regulator parameters

Let us focus on the DO-control system and choose a more limited plotType. We study the impact of PI control parameters and see if we can decrease the control error without loosing stability.

```
In [14]: # Let us take a closer look at the DO-control system and try to make control error
newplot(title='Yeast fedbatch cultivation - DO-control - increase K', plotType='
for value in [10, 20, 40]: par(K=value); simu(20)

# Reset K to the original value
par(K=10)
```



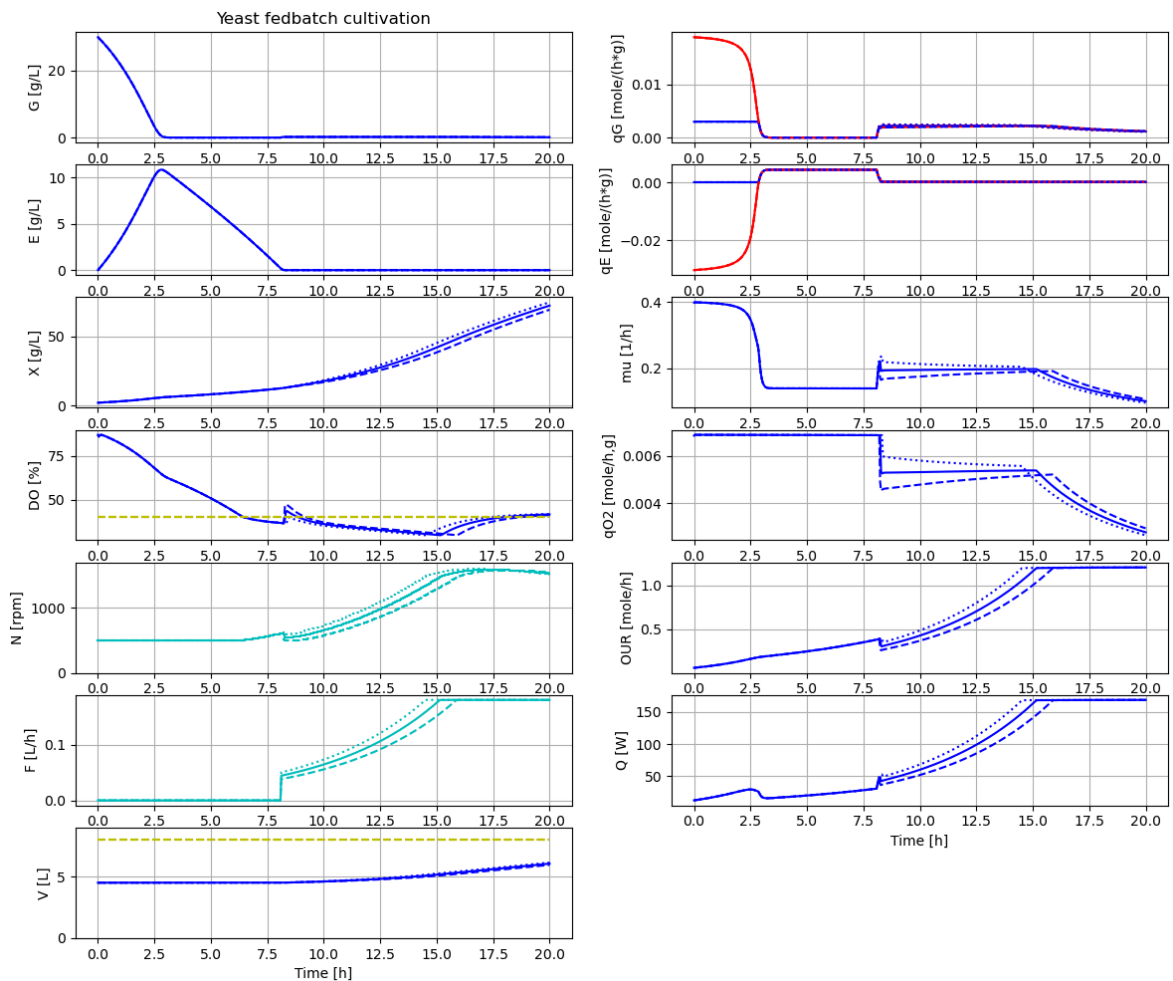
We see that by a higher control gain K the DO-control error get smaller and the stability of the control system is maintained.

Exercise I leave for you to study the impact variation of the T_i -parameter. Just make a new cell below. Then copy and paste the cell above and change parameter to T_i .

Sensitivity to changes in feed-profile

Now, let us focus on investigating impact of changes in the feed-profile. The goal is to increase the produced cell mass without accumulation of by-product ethanol. Simulation can bring some insight into how behaviour of the differen variables change when by-product is formed. This insight can help to interpret experimental results.

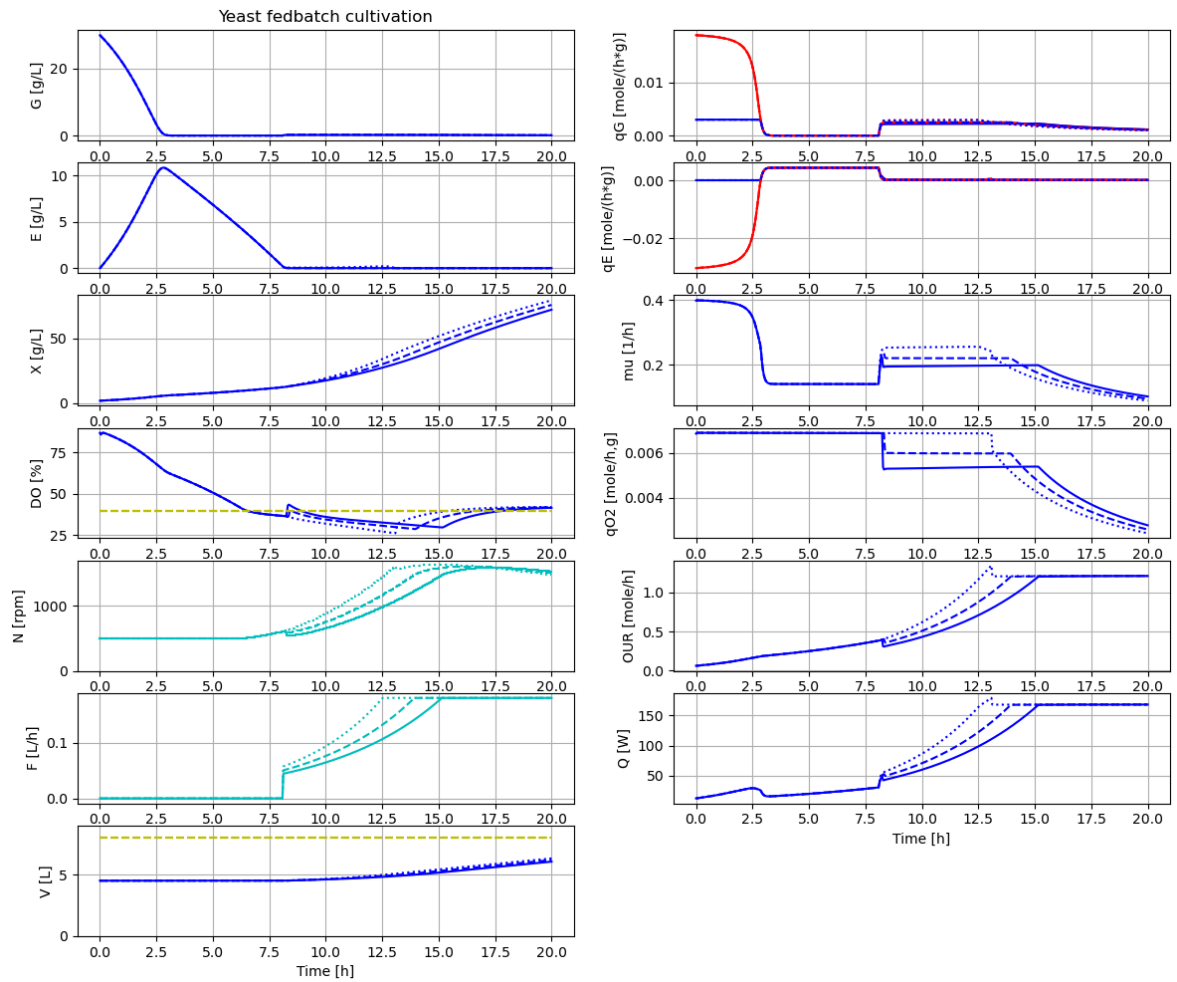
```
In [15]: # Let us check the sensitivity to changes in the feed profile design
newplot(title='Yeast fedbatch cultivation', plotType='Overview')
for value in [0.044, 0.038, 0.050]: par(F_startExp=value); simu(20)
```



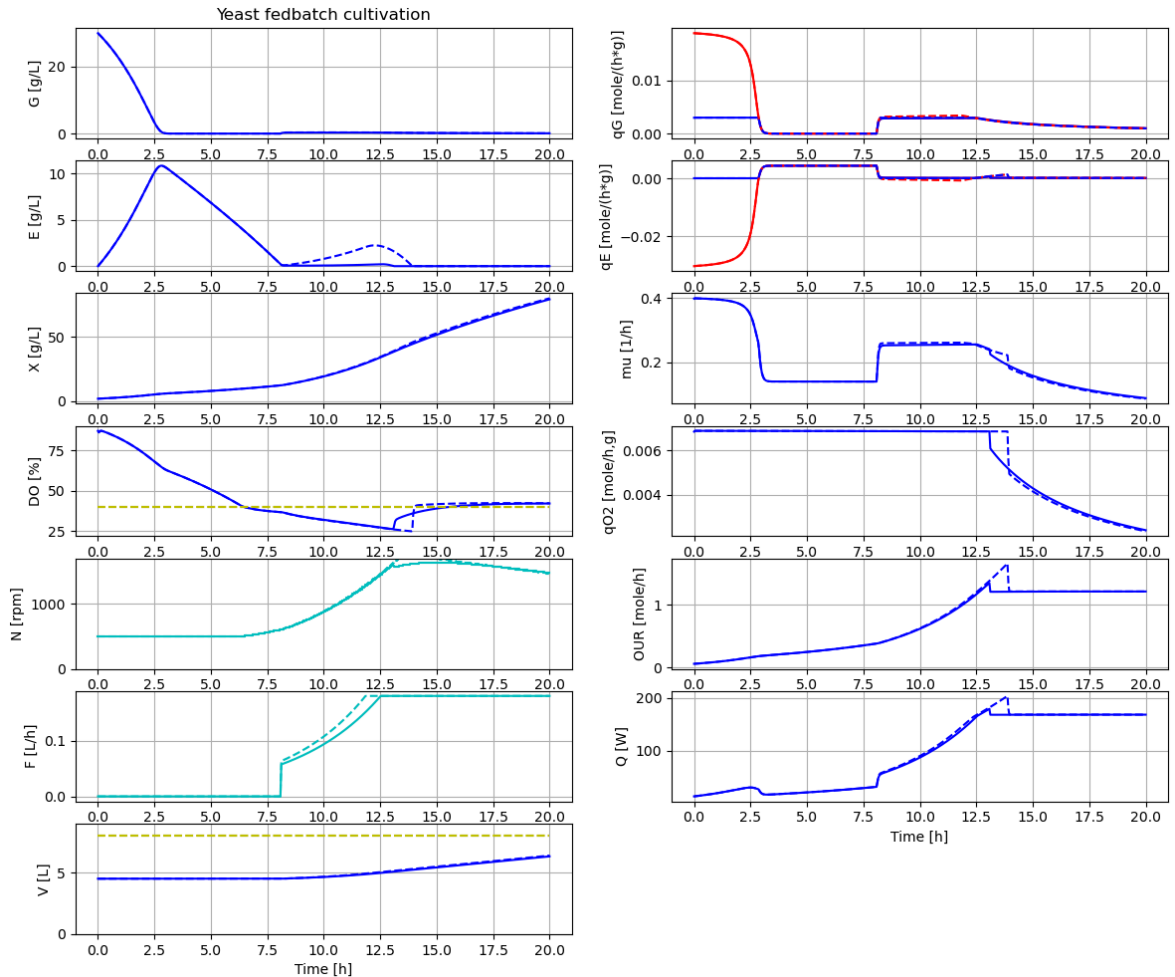
The variation in F_{start} has an impact and we see that the actual growth rate during fedbatch phase do converge to the set growth rate of the feed, but it takes more than 5 hours.

```
In [16]: # Let us investigate a feedprofile that is closer to the maximal capacity
newplot(title='Yeast fedbatch cultivation', plotType='Overview')
par(F_startExp=0.044, mu_feed=0.20); simu(20)
par(F_startExp=0.050, mu_feed=0.22); simu(20)
par(F_startExp=0.057, mu_feed=0.26); simu(20)
```

```
Out[16]: <pyfmi.fmi_algorithm_drivers.FMIResult at 0x29997645100>
```



```
In [17]: # And let us see what happens if the feedprofile exceed the culture capacity
newplot(title='Yeast fedbatch cultivation', plotType='Overview')
par(F_startExp=0.057, mu_feed=0.26); simu(20)
par(F_startExp=0.063, mu_feed=0.28); simu(20)
par(F_startExp=0.044, mu_feed=0.20)
```

Note that with the feedprofile that exceed culture respiratory capacity, ethanol is accumulated during time 8-12.5 hours. When the feedprofile then is constant from time 12.5 hours and on, then the accumulated ethanol is consumed over about an hour. This leads to a higher oxygen demand and heat production during this time. The specific cell growth rate is also slightly higher during this period.

Exercise You can investigate the impact of changing the maximal feedrate $F_{\max}$. Make sure that the DO level do not get too low.

Sensitivity to low dissolved oxygen levels

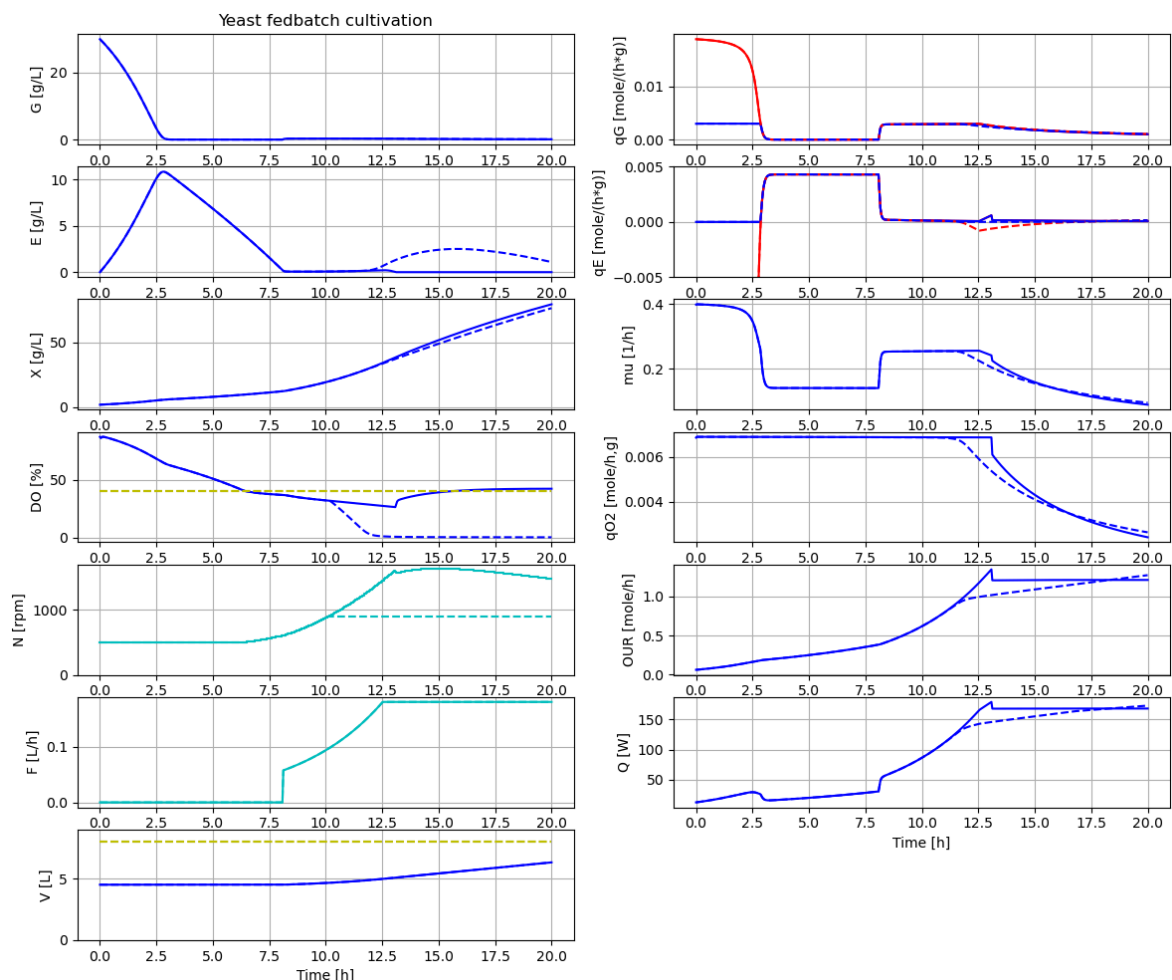
The detailed understanding of process dynamics at very low dissolved oxygen levels is not that well studied, to the authors knowledge. Here is a section on "Growth under oxygen limiation" in the original paper [1]. This model we use here. The idea is that the oxygen uptake is essentially described with a Monod-function with a parameter here denoted K_{sO_2} . At higher dissolved oxygen levels the uptake is saturated by the respiratory capacity and in our model denoted $q_{O_2\max}$. Thus, at lower dissolved oxygen levels the respiration capacity is lowered and denoted here $q_{O_2\lim}$ and at sufficiently high dissolved oxygen levels the $q_{O_2\lim}$ corresponds to $q_{O_2\max}$. The value of $q_{O_2\lim}$ controls metabolism and growth.

$$q_{O_2\lim} = q_{O_2\max} \left\{ \frac{c_{[O_2]}}{K_{sO_2} + c_{[O_2]}} \right\}$$

The process dynamics for variation of glucose feed rate around the respiratory capacity under well-aerated conditions are well described in [3]. It would be interesting to make similar studies at low dissolved oxygen levels. Also interesting to describe the impact of variation in the aeration during condition of a constant glucose supply to see the impact of ethanol production and consumption this way.

During the time 1980-90 there was an interest to study the impact variation in dissolved oxygen in two-reactor setups [5]. In the smaller reactor dissolved oxygen level was low while higher in the larger reactor and the culture was circulated in the system at a rate related to typical mixing times in a large reactor. This experimental setup has been simulated with Bioprocess Library with focus on substrate gradients rather than oxygen gradients [6].

```
In [18]: # Let us instead see what happens when the aeration read a limit in terms of sti
newplot(title='Yeast fedbatch cultivation', plotType='Overview'); ax[3].set_ylim
par(KsO2=1e-5)
par(F_startExp=0.057, mu_feed=0.26); simu(20)
par(F_startExp=0.057, mu_feed=0.26, N_high=900); simu(20)
par(F_startExp=0.044, mu_feed=0.20, N_high=2000)
```



We see that after 10 hours the maximal stirrer speed is reached and that limits oxygen transfer and dissolved oxygen goes down. At about 12.5 hours the dissolved oxygen has come down to low levels that limits the specific oxygen uptake rate and lower than the specific respiratory capacity and we get ethanol production.

Summary

- We have first seen an overview diagram of a typical yeast fedbatch cultivation where the feed started about an hour after the batch phase was finished. A new simulation was made where the feed started directly after detection of lack of substrate.
- We also took a look at the DO-control system and saw that we could decrease the control error by increasing the PI-controller gain. Stability of the control system remained.
- Then we tested variations in the feed dosage scheme and investigated the possibilities to increase the production.
- We saw what happens if the feed dosage exceeds the culture respiratory capacity and what to look for during the experimental work.
- We also saw what happens if the aeration is limiting and the tendency of the culture to produce ethanol.

References

[1] Sonnleitner, B. and O. Käppeli "Growth of *Sacharomyces cerevisiae* is controlled by its limited respiratory capacity: formulation and verification of a hypothesis", Biotech. Bioeng., 1986.

[2] von Stockar, U., Gustafsson, L., Larsson, C., Marison, I., Tissot, P. and Gnaiger E. "Thermodynamic considerations in constructing energy balances for cellular growth", Biochimica et Biophysics Acta, vol 1183, p 221-240, 1993.

[3] Axelsson, J. P. "Experimental techniques and data analysis to determine baker's yeast ethanol dynamics", Anal. Chim. Acta, vol 213, p 151-163, 1988.

[4] Axelsson, J. P., C. F. Mandenius, O. Holst, P. Hagander and B. Mattiasson "Experience in using an ethanol sensor to control molasses feed-rates in Baker's yeast production", Bioprocess Engineering, vol 3, p 1-9, 1988.

[5] Sweere, A. P. J., Matla, Y. A., Zandvliet, J., Luyben, K. C. A. M., Kossen, N. W. F., "Experimental simulation of glucose fluctuation", Appl. Microbial. Biotechnol., Vol 28, pg 109--115, 1988.

[6] Axelsson, J.P., "Investigation of effects of a hot-spot in bioprocess control", oral presentation, Nordic Process Control Workshop, Sigtuna, Sweden, August 24-26, 2016.

Appendix

```
In [19]: disp('culture', decimals=5, mode='long')
```

```
bioreactor.culture.qGmax : qGmax : 0.02
bioreactor.culture.Ks    : Ks    : 0.01
bioreactor.culture.qO2max : qO2max : 0.0069
bioreactor.culture.KsO2  : KsO2  : 1e-05
```

```
In [20]: describe('q02lim', decimals=5)
```

Limitation of respiratory capacity : 0.00263 [mole/(g*h)]

```
In [21]: # Specific growth rate at the end of the last cultivation
describe('mu')
```

Cell specific growth rate variable : 0.094 [1/h]

```
In [22]: disp('PIDreg', mode='long')
```

```
PIDreg.K : K : 10.0
PIDreg.Ti : Ti : 0.5
PIDreg.Td : Td : 0.0
PIDreg.Nd : Nd : 3.0
PIDreg.I_start : I_start : 0.0
PIDreg.D_start : D_start : 0.0
```

```
In [23]: # List of components in the process setup and also a couple of other things like
describe('parts')
```

```
['airFlow_setpoint', 'atmosphere', 'bioreactor', 'bioreactor.culture', 'bioreactor.gas_liquid_transfer', 'DO_setpoint', 'dosagescheme', 'DOSensor', 'feedtank', 'gasphase', 'gastube', 'liquidphase', 'MSL', 'N_high', 'N_low', 'PIDreg']
```

```
In [24]: describe('MSL')
```

MSL: RealInput, RealOutput, LimPID-components

```
In [25]: system_info()
```

```
System information
-OS: Windows
-Python: 3.12.11
-Scipy: not installed in the notebook
-PyFMI: 2.21.0
-FMU by: JModelica.org
-FMI: 2.0
-Type: FMUModelCS2
-Name: BPL_YEAST_AIR.Fedbatch_DOcontrol
-Generated: 2026-04-09T22:04:45
-MSL: 3.2.2 build 3
-Description: Bioprocess Library version 2.3.2
-Interaction: FMU-explore version 1.1.5
```

```
In [26]: SDG('explain')
```

"Soli Deo Gloria"
It is latin and means "To the honour of God".
The great composer Johan Sebastian Bach used to end his compositions with this small remark SDG.
And I like to do that too :).

```
In [ ]:
```